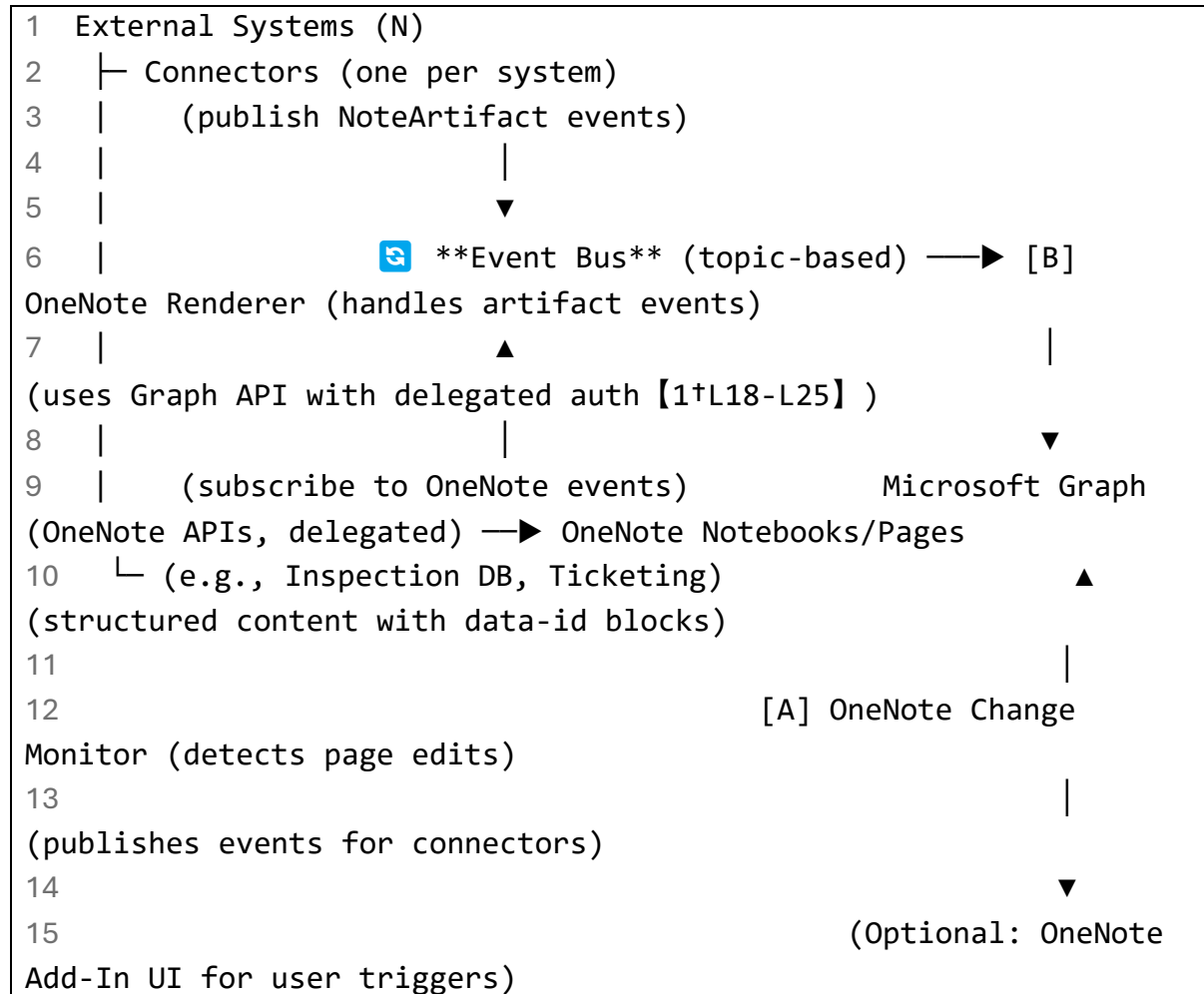


1. Reference Architecture Diagram (Textual)

Overview: The integration platform is composed of loosely coupled components communicating via an event bus. Below is a text-based diagram of major components and data flow, emphasizing bidirectional sync and delegated auth:



Component Descriptions:

- **Connectors:** Lightweight adapters for each external system. They poll or subscribe to source changes and translate them into canonical **NoteArtifact** events. Connectors publish events (artifact.created/updated/deleted) to the **Event Bus** and also listen for OneNote-originated changes relevant to their system (for bidirectional sync). Each connector is independent; adding/removing one doesn't affect others due to the decoupling by events.
- **Event Bus:** A central pub/sub message bus (e.g., Azure Service Bus or similar) that routes events between producers and consumers. It carries events like artifact.created.v1, artifact.updated.v1 (from connectors) and onenote.page.edited.v1 (from OneNote monitor) among others. Loose coupling via the bus ensures each component only interacts via standardized event contracts, enabling extensibility and isolation.
- **State & Mapping Store:** A database or persistent store that tracks:

- ID mappings between external records (artifactId) and OneNote pages (pageId).
- Sync cursors or timestamps per connector (to know where to resume).
- Content hashes or version markers for each block of a OneNote page (to detect changes and conflicts).
- Possibly stores user token/credential details if using a service account or per-user tokens. This store is accessed by connectors, the OneNote renderer, and the change monitor to maintain idempotency and consistency.
- **OneNote Renderer:** A service that subscribes to NoteArtifact events from the bus and is responsible for **applying those changes into OneNote**. On an artifact.created event, it creates a new OneNote page (in the appropriate Notebook/Section as configured) with content from the canonical object. On updates, it uses the OneNote Graph API to **PATCH** specific page elements by data-id (see Block Contract below), ensuring only the intended sections are updated 【3†L33-L41】 【4†L59-L67】 . The renderer operates with **Microsoft Graph delegated authentication**: it acts on behalf of a user or service account to call Graph (by design, Graph requires delegated tokens for OneNote operations after Mar 31, 2025 【1†L18-L25】). The renderer never calls external systems directly; it only processes events.
- **OneNote Change Monitor:** A service (or set of background functions) responsible for detecting changes in OneNote pages and translating those into events for connectors. This could be implemented via Microsoft Graph webhooks on OneNote content or via periodic polling of known pages (checking lastModifiedTime and content hashes). When a page that corresponds to a known artifactId is edited (especially if a user manually edited an “owned” block or added notes), the monitor publishes an event like onenote.page.edited with details of which artifactId changed and which block(s) were affected. Connectors listening to these events can then fetch the updated content (from Graph) and apply changes back to their source system if needed (achieving bidirectional sync). The monitor uses the same Graph delegated auth context and respects the block ownership rules (it can identify which parts of the page belong to which integration via data-ids).
- **Microsoft Graph (OneNote API):** The platform uses Microsoft Graph’s OneNote endpoints (e.g., /me/onenote/... or /sites/{site-id}/onenote/... depending on notebook location) for all OneNote access. All Graph calls use **delegated permissions** (app+user) because OneNote APIs do not support app-only tokens post-2025 【1†L18-L25】 . This means Graph calls execute in the context of a user identity: either a dedicated service account that owns the notebooks or the actual end-user (with prior consent). Graph operations include creating pages,

updating page HTML, uploading attachments, and possibly subscribing to change notifications.

- **OneNote Notebooks/Pages:** The actual OneNote notebooks (could be personal notebooks, shared team notebooks, or notebooks owned by a service account). Within notebooks, pages created by the integration contain content divided into identifiable sections (blocks) with stable IDs. Users can view and edit these pages in OneNote (any client). The content structure and block IDs are dictated by the templates (see Block Contract below). Users are free to add their own notes outside the managed blocks, and the system will preserve those.
- **(Optional) OneNote Add-In:** An Office JavaScript add-in that can be embedded in OneNote (primarily OneNote web or possibly desktop) to enhance user interaction with the integration. This add-in might provide UI elements like “Link this page to a record” or “Refresh from source” buttons. It operates in the user’s context and may call the integration’s APIs or Microsoft Graph directly (with the user’s token) to trigger on-demand sync actions. It is not required for core functionality but is part of the extensibility for user-initiated workflows (for example, to handle conflicts or to allow the user to choose a notebook/section for new integrations).

Loose Coupling Emphasis: All external systems communicate only through the canonical events and the **NoteArtifact** data model. The OneNote renderer and change monitor are generic – they do not have hardcoded knowledge of specific external systems, they only know how to handle the canonical format and interact with OneNote/Graph. This means new connectors can be added by producing the same events, and the rest of the system will handle them. Similarly, removing a connector simply stops its events – no other component fails as a result.

Bidirectional Sync Flow: In summary, for a create/update from an external system, a NoteArtifact event flows [External System] → Connector → Event Bus → OneNote Renderer → Graph → OneNote. For a user edit or OneNote-side change, the flow is [OneNote] → Graph/webhook → OneNote Change Monitor → Event Bus → Connector(s) → [External System]. All flows consult the State Store for mappings and use the Canonical schema.

2. Canonical Schema (NoteArtifact) and Block Contract

Canonical Schema (JSON): We define a JSON schema for the versioned **NoteArtifact** object (let’s call the current version **v1**). This schema is used as the common interchange format on the Event Bus, ensuring each connector and the OneNote renderer speak the same language. Below is the schema structure with brief explanations:

```

1  {
2    "version": "1.0",
3    "artifactId": "string",          // Unique ID for the note
artifact (primary key; stable across updates)
4    "sourceSystem": "string",        // Name/key of the source
system (e.g., "InspectionsApp", "Jira", "Obsidian")
5    "sourceRecordId": "string",      // Identifier or URL pointer
to the source record (if applicable)
6    "title": "string",              // Title or heading for the
note
7    "body": "string",               // Main content (rich text,
markdown or HTML fragment)
8    "tags": ["string"],             // Tags or categories
associated
9    "people": [                     // People or stakeholders
linked to this note
10     {
11       "name": "string",
12       "role": "string"            // e.g., "author",
"assignedTo", etc., if relevant
13     }
14   ],
15   "attachments": [                // List of file attachments
or images related to the note
16     {
17       "filename": "string",
18       "fileId": "string",          // ID or path if stored in
OneNote or external storage
19       "url": "string"              // (optional) link to file
if accessible
20     }
21   ],
22   "timestamps": {                 // Timestamps for record
keeping
23     "created": "2026-04-20T18:32:00Z",
24     "lastUpdated": "2026-04-20T19:10:00Z"
25   },
26   "relationships": {              // (optional) Links to other
artifacts or hierarchy info
27     "parentArtifactId": "string",
28     "relatedArtifactIds": ["string"]

```

```

29     },
30     "routing": {                                // Routing hints for OneNote
Renderer
31         "notebookName": "string",
32         "sectionName": "string",
33         "template": "string"                    // Which template to use for
page layout
34     },
35     "renderHints": {                            // Presentation preferences
36         "layout": "string",                    // e.g., "timeline",
"detailed", etc., or template variant
37         "priority": "high"                     // e.g., could influence
highlighting in UI
38     }
39 }

```

Notes on Schema Fields:

- Fields like **artifactId**, **sourceSystem**, **title**, **body** are typically required. Others can be optional depending on context. For instance, an Obsidian note might not have “people” or “attachments” initially.
- **version**: Allows evolving the schema (for future v2, etc.). Connectors and renderers must handle at least one version and can ignore fields they don’t recognize for forward compatibility.
- **sourceRecordId**: Links back to the external system’s record (could be an ID or a deep link URL). This can be used to provide backlinks in the OneNote page (e.g., “Open in Jira” link).
- **body**: Ideally a normalized content format (possibly HTML snippet or markdown) that the OneNote renderer will insert into the page. (Connectors may produce markdown which the renderer can convert to HTML, or produce HTML directly.)
- **attachments**: May include pointers to files; the OneNote renderer would have to actually upload or embed these via Graph (Graph supports sending multipart content for images/attachments).
- **routing**: Decides where and how to create the page in OneNote. The logic that interprets this might be configurable (e.g., map `sourceSystem == "Jira"` to a specific notebook or section). “template” refers to a layout template name defined in our Template Registry (outside this artifact).
- **renderHints**: Additional directives for how to format the note (could be flags for the template, or visual cues like priority or status that the OneNote page might reflect e.g., with color-coding or an icon).

Block Contract (OneNote Page Structure & Ownership):

To safely support multiple integrations and user edits on the same page, the OneNote content is divided into **identifiable blocks** that are individually addressable and owned. The “block contract” defines how these blocks are structured:

- Each **OneNote page** created by the system is essentially a template filled with content from the NoteArtifact. The template is divided into named sections, each with a unique HTML data-id attribute. For example, a template might have:
 - A **Title block** (OneNote page title) – this is separate from page content in OneNote API, but conceptually it’s a block (Graph API refers to it via the title target).
 - A **Header or Summary block** summarizing the item (data-id="summary").
 - A **Details/Body block** with the main content (data-id="details").
 - A **Source Info block** with metadata or links back to the source (data-id="source-info").
 - An **Attachments block** listing files (data-id="attachments").
 - Optionally, blocks for **tags** or **people** (e.g., data-id="tags", data-id="people").
 - A reserved area for **User Notes** (which could either be not tagged with a data-id, or given a special data-id like "user-notes" that the system knows *not* to overwrite).
- **Example Page HTML Structure:** (simplified for illustration)
- In this example, everything inside data-id="..." blocks (summary, details, source-info, attachments, people) is populated and owned by the integration. The user-notes block is a section intended for user edits – the system will never overwrite it (in fact, it could omit data-id for user notes to treat it as ad-hoc content). The **page title** ("Incident #1234") is set from NoteArtifact.title.
- **Ownership & Update Rules:** Each integration is responsible for certain data-id blocks:
 - The system guarantees that **only one component/connector will write to a given** data-id. For example, if the “source-info” block is managed by the connector for ExternalApp, no other connector will touch that block. This prevents conflicts between integrations.
 - When an external update occurs (e.g., the incident description changed in ExternalApp), the connector publishes an update event, and the OneNote renderer will PATCH the OneNote page, targeting only the #details block’s content with the new HTML. Other blocks on the page remain unchanged. Microsoft Graph’s OneNote API supports targeting specific elements by data-id for replacements **【3†L33-L41】** **【4†L59-L67】** .

- If the user edits content *inside* an owned block (for example, fixes a typo in the details section via OneNote), the OneNote Change Monitor will detect that change (by comparing content hashes) and can either flag a **conflict** or attempt to merge it. In a simple implementation, we might consider any user edit in a system-owned block as a conflict that requires attention – because the source system still has the old text. A more advanced approach could attempt to update the external system, but that’s domain-specific (e.g., pushing a user’s typo fix back to the ExternalApp’s description field).
- If the user adds entirely new content (like a new paragraph) outside of any recognized data-id block, that content is considered user-owned and will be left untouched by the system. We won’t delete or overwrite it on subsequent syncs. (If needed, the OneNote renderer can always append new blocks *below* any user content to avoid disrupting their notes.)
- **Templates and Data-Ids:** The structure of blocks (which blocks exist, with what data-ids) is defined by a template (or by convention for each source type). Templates ensure that for a given type of NoteArtifact, the page will have a known layout. Multiple templates might exist for different sources or record types, but all must adhere to the block contract principle. If a template is updated (say we add a new block), the renderer and connectors should ideally handle it gracefully (older artifacts might not have it, that’s fine).
- **Block Identification in Code:** When the OneNote renderer gets a NoteArtifact, it first finds or creates the OneNote page, then ensures all required blocks exist (creating any missing ones via Graph PATCH insert actions). It uses data-id selectors for targeting. For example, if updating attachments, it might send a PATCH with target: "#attachments" and action: replace with new ... content. Graph will replace the content of that div only. This fine-grained update mechanism prevents us from accidentally overwriting user edits that are in other blocks.
- **Consequence of Contract:** This design permits multiple integrations on one page (rare but possible) and safe user collaboration. For instance, if two connectors output to the same page (maybe a “summary” from System A and “metrics” from System B, on one combined dashboard page), each will have separate blocks. Users can annotate around those blocks. Each connector only touches its part. The page becomes a living document that combines automated data with human commentary.

3. Engineering Decision Record (ADR): Delegated Authentication & Ownership Model

- **ADR Title:** Use of Delegated Graph API Authentication with a Service Account for OneNote Integration
- **Status:** Accepted (2026-04-20)
- **Context:** Our OneNote integration platform needs to programmatically access and modify OneNote notebooks and pages via Microsoft Graph. Originally, one might use *app-only* Graph API permissions for a background service. However, Microsoft announced the retirement of app-only authentication for OneNote APIs effective March 31, 2025 **【1†L18-L25】**. After this date, any Graph calls to OneNote must use **delegated permissions**, meaning an actual user context is required for auth. This change is driven by security concerns and is now a hard constraint. Therefore, we must choose a delegated authentication strategy. Additionally, we must decide **whose** user context to use for operations – i.e., an application-specific service account vs. each end-user’s identity. This decision has implications for data ownership (which account “owns” the created OneNote pages) and complexity of implementation.
- **Decision: We will adopt a Service Account Delegated Auth model.** The integration will use a dedicated Microsoft 365 user account (or a small number of accounts, one per department or use-case) as the identity for all Graph calls. In practice, this means:
 - a. An admin will grant tenant-wide consent for the application’s delegated permissions (OneNote notebooks read/write) on behalf of the service account(s).
 - b. Our backend will maintain an OAuth refresh token (or client certificate) for the service account to authenticate and obtain delegated tokens without interactive login after initial setup.
 - c. All OneNote pages created by the integration will be owned by this service account (e.g., in notebooks that either belong to the service account or are shared with it). We may use a single centrally-located Notebook (e.g., a notebook on a SharePoint site or owned by the service account) to store integration pages, or potentially separate notebooks per team. But in all cases, the modifications are done under the delegated identity of the service account.
- **Rationale:** The service account approach offers a controlled context: we avoid the complexity of per-user tokens while still complying with Microsoft’s delegated auth requirement. It centralizes OneNote content management, which is appropriate since the integrated data often constitutes a team or organization knowledge base rather than private personal notes. It’s also easier to implement initially (one identity to manage) and aligns with the way our platform treats OneNote as a shared resource.
- **Options Considered:**

- a. **Per-User Delegated Access:** Each user would individually authorize the integration (via OAuth) to access their OneNote. The integration would then act on behalf of many users, storing tokens for each. *Pros:* Each user's notebooks could be used, respecting personal ownership and security scoping; no single account has blanket access. Good for personal integrations (e.g., each user syncing their own notes). *Cons:* Considerably more complex to implement and maintain (token management at scale, user onboarding/offboarding flows); difficult to ensure consistency if a user hasn't signed in or consented; if the integration needs to share a page among multiple users, it gets complicated with permissions.
 - b. **Single Service Account (Delegated):** (*Chosen*) Use one dedicated account (with a real M365 license) as the OneNote content owner. *Pros:* Simple model – only one user context to manage; easy to swap out if needed; all pages are in one place (or under one owner) which simplifies locating content. This account can be given a descriptive name (e.g., “OneNote Integration Bot”) to clearly indicate automated content. *Cons:* Puts all eggs in one basket – if the account credentials fail or the account is deleted, the integration halts; all notebooks/pages are technically owned by a single user which might not align with data ownership policies; also, that account needs appropriate access (if pages are in a team notebook, the account must have permissions). Security-wise, this account becomes powerful; it must be well protected.
 - c. **Service Account per Department/Use-case:** A variation where multiple dedicated accounts are used, e.g., one per integrated system or department. *Pros:* Limits blast radius of any single token compromise; segregates data ownership somewhat (e.g., HR notes owned by HR bot, IT notes by IT bot). Could help with per-application throttling limits as well by splitting load. *Cons:* More accounts to manage, but still fewer than per-user; need clear mapping of which account handles which data.
 - d. **App-Only (Client Credentials):** Use client credential flow to get an application token. *Pros:* Easiest to implement (no interactive auth, no user context needed). *Cons:* **Not viable** due to Graph OneNote app-only deprecation **【1†L18-L25】** . Also, even before deprecation, app-only had broad access that could be risky and was not granular.
- **Consequences of Decision:**
 - a. **Delegated Auth Implementation:** We will set up the chosen service account(s) and go through an OAuth consent flow once per account. Because it's delegated, a Global Admin will have to consent to the required permissions for the app (with possibly “Admin consent” so it doesn't require each user). We must securely store the tokens (likely an

OAuth refresh token or client secret for the service account) in our configuration (e.g., Azure Key Vault). We also need a mechanism to renew tokens before expiration. Running this as a background service means using something like the OAuth 2.0 ROPC flow (resource owner password credentials) is not recommended for production, so instead we'll likely use the standard auth code flow with a one-time manual sign-in to seed a token, or a client certificate mapped to the service account (if using on-premises ADFS or similar capabilities).

- b. **OneNote Content Ownership:** All OneNote pages will show the service account as the author/last editor when automated changes occur. Team members will need access to the notebooks/pages. If using a notebook on SharePoint or O365 Group, we must invite the relevant real users (or a security group) to that notebook. If the service account owns a personal notebook, it will share sections with others as needed. This is an extra admin step but once set, content is centrally accessible.
- c. **Auditing and Compliance:** Using a single account means actions are not tied to the real human who triggered them (if any). We should ensure that major changes are logged in our own audit logs (so we know which connector or external event caused a OneNote page update, since from Microsoft's side it will just show "service account made an edit"). This helps with traceability.
- d. **Rate Limits:** Microsoft Graph applies throttling limits per app and per user. With one user account doing all calls, we have to be mindful of not hitting OneNote's API call limits. If we approach limits, we might implement scaling (multiple accounts) or request higher throttling quotas from Microsoft. Alternatively, distributing integration load across multiple service accounts (option 3) could be a future scaling strategy if needed.
- e. **User Experience Implications:** In the service account model, end-users do not need to individually sign in or manage tokens, which streamlines adoption. However, they also can't on their own integrate new notebooks from their personal space without involving the service (since the service account won't have access to those by default). This is acceptable because our use-case centralizes on shared knowledge bases.
- f. **Security Mitigations:** We will:
 - Treat the service account's credentials with high sensitivity (vault storage, limited access).
 - Possibly enable conditional access rules to restrict where this account can log in from (only allow our integration server's IP or require MFA for interactive login).
 - Monitor the account's activity (unusual patterns, failed login attempts, etc.).

- Have break-glass access in case the account is locked (e.g., a second admin account that can grant it access again or an alternate).
- **Related Decisions / Future Considerations:**
 - a. We may revisit this decision if Microsoft introduces a more secure app-only method or a managed identity approach for Graph OneNote in the future, but as of now, delegated is the only path.
 - b. If down the line we decide to allow personal integrations (users syncing their own stuff beyond the central systems), we might implement the hybrid approach: keep the service account for core automated publishing, and optionally use per-user auth for user-specific notebooks (with a user-facing onboarding UI). The architecture is flexible enough to add that later, but it would complicate the mapping of artifact to owner.
 - c. We should also decide where the notebooks will live: options include a dedicated SharePoint site (with OneNote sections) vs. the personal notebook of the service account. A SharePoint site (perhaps named “Integration Notebook Hub”) might be better for multi-user access and backup purposes (since multiple owners can be set for the site). This detail will be addressed in the implementation tasks.

ADR Conclusion: We choose a delegated auth model using a service account, acknowledging the trade-offs. This approach meets the immediate requirement (compliance with Graph’s delegated auth mandate 【1†L18-L25】) and provides a simpler, controllable integration point. We will implement the necessary safeguards and be prepared to adjust if usage patterns or scale demands a change (e.g., introducing additional service accounts or per-user tokens for specific cases).